Model 3 — Adding Recency Features

M5 Retail Demand Forecasting — Problem Statement 11

Testing the EDA's strongest reported signal

NPN AIA Hackathon — St. Joseph's College of Engineering

Generated 2026-08-14 from executed run `model_03_tweedie_recency`.

***Terms used in this report. RMSE** (Root Mean Squared Error) measures how far predictions land from actual sales, counting big misses much more heavily than small ones — lower is better. **MAE** (Mean Absolute Error) is the plain average size of the miss. **WAPE** expresses total error as a share of total actual demand, which exposes a model that scores well simply by predicting near-zero everywhere. **Leakage** means letting information into the model that would not have existed at the moment the forecast was really made. **SNAP** is the US Supplemental Nutrition Assistance Program, a food-assistance benefit; the dataset records, per state per day, whether it was usable. **Intermittent demand** describes a product that sells on some days and records zero on many others.*

Objective

The EDA identified recency as the cleanest relationship in the entire dataset: the chance of a sale today falls from 65.2% if the item sold yesterday to 0.6% after 29 or more days without a sale. This experiment tests whether encoding that as explicit features improves a 28-day forecast.

What we did

Model 2 plus feature group C: `days_since_last_sale`, `zero_streak_length` and `days_since_first_sale`. Nothing else changed.

One thing established in the foundation stage is worth repeating here: at a fixed forecast origin `days_since_last_sale` and `zero_streak_length` are **the same number**. If the last sale was three days ago then there are exactly three consecutive zero days ending at the origin. They were both built because the specification asked for both, but they are perfectly correlated.

Data used

Training rows	12,805,800
Training origins	15 (each contributes 30,490 series x 28 days)
Features	30
Feature groups	A, B, C, E, F, G
Feature set	`base_recency` — BASE + Recency

Training origins are spaced 28 days apart, so their 28-day target blocks tile the history contiguously and no (series, day) target is counted twice.

Model configuration

Setting	Value
Model	LightGBM
Objective	tweedie (variance_power=1.1)
learning_rate	0.05
num_leaves	128

Setting	Value
max_depth	-1
min_data_in_leaf	100
feature_fraction	0.8
bagging_fraction	0.8
lambda_l2	1.0
seed	42
Boosting rounds	400
Categorical features	12 handled natively by LightGBM
Random seed	42
Training time	114.9s

No early stopping was used, deliberately. *Stopping when the validation score stops improving would let the validation window influence a training decision, and the resulting score would flatter itself. A fixed number of rounds keeps the held-out estimate honest.*

Validation design

Every model in this project is scored on exactly the same window, with the same metric code, so the comparisons between them are fair by construction.

Forecast origin	d_1913
Days predicted	d_1914 .. d_1941
Dates predicted	2016-04-25 .. 2016-05-22
Horizon	28 days
Series	30,490
Predictions scored	853,720

The model sees no sales at all from the validation window. It is given only the calendar, event, SNAP and price information for those days, all of which are genuinely published in advance.

Results (measured)

Metric	Value
RMSE	**2.1258**
MAE	**1.0320**
WAPE	0.7153
Bias (mean predicted – mean actual)	-0.0763
Predictions scored	853,720

Comparison against the previous step

	RMSE	MAE
Model 2 — Tweedie, no recency	2.1256	1.0315
This model	**2.1258**	**1.0320**
Change	+0.0003	+0.0006

Measured verdict: this change **made no meaningful difference**.

What the model learned

This is the experiment where the project's expectations and its measurements part company, and the honest reading is that the **relationship being real is not the same as the feature being useful**. The dry-spell pattern the EDA found is genuine — but the rolling means and lags already in Model 2 carry essentially the same information. A series with `rolling_mean_28 = 0` is, by definition, a series in a long dry spell. Adding an explicit counter tells the model something it could already deduce.

Leakage checks

The foundation stage established the guarantee this model inherits, and it was verified by experiment rather than asserted: every sales value after the forecast origin was overwritten with an absurd number (9999), the features were rebuilt, and all 32 came back **bit-for-bit identical**. A companion check confirmed the target column did change, proving the corruption really reached the data.

In addition, the training-set builder refuses to run if any training row targets a day inside the validation window — that is an assertion in code, not a convention.

Limitations

- Recency was tested as additional features on a global model. A different architecture (for example one that conditions on dry-spell state directly) might use the signal differently.
- The two recency features are mutually redundant, which splits any importance they do have across duplicate columns.

Conclusion and next step

Measured result: adding recency made no meaningful difference. It is retained for the next experiment only so that Model 4 tests the listing group on top of an unchanged base, but it is not carried forward as a claimed contribution.